

Numerical Analysis of the SIR Epidemic Model: Solver Accuracy, Parameter Recovery, and Robustness to Noise

CSE 402: Numerical Analysis, Simulation & Modeling — Section B, Group 2

Saif Uz Zaman
Bangladesh University of
Engineering and Technology
Dept. of CSE — Section B, Group
2 — ID 2105088
Dhaka, Bangladesh

Sayaad Muzahid Masfi
Bangladesh University of
Engineering and Technology
Dept. of CSE — Section B, Group
2 — ID 2105066
Dhaka, Bangladesh

Ibtida bin Ahmed
Bangladesh University of
Engineering and Technology
Dept. of CSE — Section B, Group
2 — ID ?
Dhaka, Bangladesh

Sakif Naieb Raiyan
Bangladesh University of
Engineering and Technology
Dept. of CSE — Section B, Group
2 — ID ?
Dhaka, Bangladesh

Aurchi Chowdhury
Bangladesh University of
Engineering and Technology
Dept. of CSE — Section B, Group
2 — ID ?
Dhaka, Bangladesh

Abstract

Parameter estimation for the SIR epidemic model is typically studied with the ODE solver treated as a black box. We make solver choice the central experimental variable, implementing Forward Euler, Heun’s method, classical RK4, adaptive Dormand–Prince RK45, and two implicit methods from scratch, and asking how solver accuracy propagates into least-squares-recovered transmission and recovery rates (β , γ). Using a semi-analytic gold-standard SIR solution (derived from the model’s exact phase-plane reduction) as ground truth, we show that estimation bias inherits the fitting solver’s convergence order, and we locate a crossover noise level σ^* at which this solver-induced bias is exactly matched by noise-induced estimator variance: below σ^* , solver choice dominates the error budget; above it, it is statistically irrelevant. The pipeline draws on all four CSE 402 lab assignments and the optimization home assignment – root finding, LU and QR factorization, eigenvalues by the power method, random number generation and Monte Carlo simulation, optimization, least squares, interpolation, and quadrature. We map each method to where it is used and document how every random number is generated. All code, experiments, and figures in this report are regenerated from a single reproducible pipeline (`make all`).

Keywords

SIR epidemic model, ODE solvers, parameter estimation, uncertainty quantification, Monte Carlo simulation, numerical analysis

1 Introduction and Base Paper Review

The SIR (Susceptible–Infectious–Recovered) model is the canonical compartmental description of an epidemic, and fitting it to case data is a standard tool for estimating a pathogen’s transmission rate β and recovery rate γ , and

hence its basic reproduction number $R_0 = \beta/\gamma$. Capaldi et al. [2] study parameter estimation and uncertainty quantification for this model in detail: least-squares fitting, sensitivity of the estimates to β – γ correlation, and the effect of sampling frequency. Like most work in this area, they treat the numerical integration of the ODE system as a solved problem – a black box that produces “the” model trajectory, with no further scrutiny of *how* it is produced.

We ask the question that black box leaves closed: *does the choice and accuracy of the ODE solver materially affect the parameters recovered from it*, once realistic imperfections – observation noise, sparse sampling, an incompletely observed epidemic – are introduced? We answer it by implementing every solver ourselves (Forward Euler, Heun, classical RK4, adaptive Dormand–Prince RK45, and two implicit methods), building an estimation pipeline around them with hand-written optimizers, and running a controlled factorial experiment that isolates solver truncation error from every other source of error in the pipeline. Section 2.9 maps each method we use to the CSE 402 syllabus, and Section 2.6 documents how the random numbers behind our noise, resampling, and MCMC experiments are generated.

Our central methodological device is a *semi-analytic gold standard*: the SIR system admits an exact reduction to a single quadrature (Section 2), so we can measure every solver’s error against the true solution to machine precision, rather than against “a finer numerical solve” – which would be circular. This lets us make a quantitative claim that a black-box treatment cannot: there is a *crossover noise level* σ^* at which solver-induced estimation bias equals noise-induced estimation variance. Below σ^* , solver accuracy is the dominant source of error in the recovered parameters, and using Forward Euler at a coarse step is scientifically indefensible. Above σ^* , sampling noise dominates and a cheaper, less accurate solver is statistically indistinguishable from a more expensive one.

1.1 Base paper and our extension

Capaldi et al. [2] fit (1) to synthetic and real incidence data via least squares, quantify the uncertainty in the resulting $(\hat{\beta}, \hat{\gamma})$ via an asymptotic covariance estimate, and study how estimation quality degrades with sparser sampling. Their solver is used only as a means to an end. We take their estimation machinery as a starting point but restructure the experiment so that *solver identity and step size* are first-class independent variables, crossed with noise level, sampling interval, observation window, and the underlying (β, γ) regime.

1.2 Related numerical-methods reference

Prodanov [12] gives an analytical treatment of the SIR system's phase-plane reduction that we build on directly for our gold-standard solution (Section 2).

2 Methodology

2.1 The SIR model

The classical SIR model partitions a closed population of size N into susceptible (S), infectious (I), and recovered (R) compartments:

$$\frac{dS}{dt} = -\frac{\beta SI}{N}, \quad \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (1)$$

The basic reproduction number is $R_0 = \beta/\gamma$: an epidemic grows while $R_0 \cdot S(t)/N > 1$ and declines once susceptible depletion pushes it below 1.

2.2 Semi-analytic gold standard

Dividing dS/dt by dR/dt in (1) gives $dS/dR = -(\beta/\gamma N)S$, so

$$S(t) = S_0 \exp\left(-\frac{\beta}{\gamma N}(R(t) - R_0)\right), \quad (2)$$

an exact, solver-free relation between S and R (we write R_0 here for the initial value of the R compartment, not to be confused with the reproduction number). Substituting (2) into $dR/dt = \gamma I = \gamma(N - R - S)$ gives a *scalar* separable ODE, so

$$t(R) = \int_0^R \frac{dr}{\gamma(N - r - S_0 e^{-\frac{\beta}{\gamma N} r})}. \quad (3)$$

We evaluate (3) by adaptive Simpson quadrature and invert it by a safeguarded Newton method (the standard `rtsafe` scheme) to obtain $R(t)$, then recover S from (2) and $I = N - S - R$. This reference trajectory is accurate to machine precision using no ODE solver at all, and every accuracy claim in this paper is measured against it. We additionally use (2) to define the *phase invariant* $Q(t) = \ln S(t) + \frac{\beta}{\gamma N} R(t)$, constant along any true trajectory; unlike the trivial mass-conservation check $S + I + R = N$ (which every explicit Runge–Kutta method satisfies to roundoff at *any* step size, since the right-hand side of (1) sums to zero identically), drift in $Q(t)$ scales with the solver's true convergence order and is a genuine accuracy diagnostic.

2.3 Solvers

We implement Forward Euler (order 1), Heun's method (order 2), classical RK4 (order 4), adaptive Dormand–Prince RK45 with a PI step controller (order 5(4), used as the “truth” solver for model variants with no closed form), and two implicit methods – backward Euler and the trapezoidal rule – whose nonlinear stage equations are solved by Newton's method with the linear system at each iteration factored by our own partial-pivoting LU decomposition.

2.4 Estimation

Given noisy observations obs_k at times t_k , we minimize the weighted least-squares cost

$$J(\theta) = \sum_k w_k (\text{obs}_k - h(y(t_k; \theta)))^2 \quad (4)$$

by grid search, golden-section coordinate descent, Nelder–Mead, Gauss–Newton, and Levenberg–Marquardt, all implemented from scratch. The Jacobian required by the last two methods is obtained by integrating the *forward sensitivity equations* $\dot{s}_j = J_y f s_j + \partial f / \partial \theta_j$ alongside the state, using the same solver under test – so solver error enters the Jacobian exactly as it enters the trajectory, which is the mechanism behind our central claim, rather than by finite differences.

2.5 Eigenvalues by the power method

Every eigenvalue in the pipeline comes from our implementation of the power method as taught in the course lecture [13] (`core/sirlab/linalg/eigen.py`). Starting from $x_0 = (1, \dots, 1)^\top$, each iteration forms $y = Ax_k$, divides it by its largest-magnitude entry, and takes that normalizing factor as the eigenvalue estimate. The lecture stops once the factor's relative change $|\varepsilon_a|$ falls below a tolerance (10^{-12}). We additionally require the normalized vector to stop changing, because the factor alone can settle while x_k is still far from the eigenvector. On the lecture's 4×4 example the largest entry equals 10 from the first step, and deflating that unconverged vector would corrupt every later eigenvalue. The inverse power method runs the same loop on A^{-1} to find the smallest eigenvalue, solving $Ay = x_k$ with our LU factorization, factored once. Hotelling deflation, $A_2 = A - \lambda_1 \hat{v}_1 \hat{v}_1^\top$, then gives all eigenpairs of a symmetric matrix one at a time. Our code reproduces the lecture's three worked tables digit for digit (`tests/test_eigen.py`). We use these routines in three places:

- the axes of the asymptotic confidence ellipse, from $\text{Cov}(\hat{\theta})$ (E11, E14), and the cost-valley direction, from $J^\top J$ (E05): power method with Hotelling deflation;
- the condition number of J (E10): $\kappa_2(J)^2 = \lambda_{\max}/\lambda_{\min}$ for the eigenvalues of $J^\top J$, with $\lambda_{\max}$ from the power method and $\lambda_{\min}$ from the inverse power method;
- the stability metric $h\rho(J_y f)$, where ρ is the largest eigenvalue modulus of the SIR Jacobian (E03).

When the power method fails. The method assumes a single dominant eigenvalue, $|\lambda_1| > |\lambda_2|$, and the SIR Jacobian violates that assumption. Its eigenvalues are 0 and those of

the block $\begin{bmatrix} -a & -b \\ a & b-\gamma \end{bmatrix}$, with $a = \beta I/N$ and $b = \beta S/N$. At the epidemic peak $b = \gamma$, so

$$\lambda = \frac{1}{2}(-a \pm \sqrt{a^2 - 4a\gamma}), \quad (5)$$

a complex-conjugate pair of equal modulus whenever $a < 4\gamma$. That holds at the peak of every epidemic with $R_0 < 6.94$, including all three of our regimes ($R_0 = 1.5, 3, 6$). The normalizing factor then oscillates and never converges. When the iteration stalls like this, the iterate x still settles into the 2-D invariant subspace of the two largest eigenvalues. We orthonormalize x and Ax (Gram–Schmidt) into Q , form the 2×2 matrix $H = Q^\top A Q$, and solve its characteristic equation $\det(H - \lambda I) = 0$ as in the lecture, which gives $|\lambda| = \sqrt{\det H}$ for a complex pair. We accept the result only once $AQ = QH$ holds to 10^{-10} , i.e. once the subspace is invariant, and otherwise keep iterating. The same projection resolves a symmetric matrix whose two largest eigenvalues are λ and $-\lambda$. NumPy’s LAPACK-based eigensolvers appear only in the tests, as an independent cross-check.

2.6 Random number generation

Every stochastic step in the pipeline – observation noise, optimizer start points, bootstrap resampling, and the MCMC sampler – draws from a single pseudo-random number generator (PRNG) through NumPy’s `Generator` interface [6]. Experiments E01–E04, E12, and E14 are deterministic and draw no random numbers. We did not write the generator ourselves, so this subsection states which algorithm produces each random quantity (Table 1); together with the seeds, that makes every stochastic result reproducible.

Uniform source: PCG64. The generator that NumPy’s `default_rng(seed)` returns is PCG64 [11]. Its core is the linear congruential method of the course simulation textbook [1], run on a 128-bit state:

$$X_{i+1} = (aX_i + c) \bmod 2^{128}, \quad (6)$$

with a fixed 128-bit multiplier a and an odd increment c , which gives the full period 2^{128} . A power-of-two-modulus LCG has weak low-order bits (bit j of X_i repeats with period 2^{j+1}), so PCG does not output X_{i+1} directly. Instead it XORs the two 64-bit halves of X_{i+1} and rotates the result right by the top six bits of the state (the “XSL-RR” output permutation), giving a 64-bit integer Y_i . A uniform double on $[0, 1]$ is its top 53 bits, $U_i = \lfloor Y_i/2^{11} \rfloor \cdot 2^{-53}$, and a draw from $U(a, b)$ is $a + (b - a)U_i$. PCG64 passes the TestU01 BigCrush battery [11]; we did not rerun statistical tests such as the chi-square or Kolmogorov–Smirnov uniformity tests [1] ourselves.

Seeds and independent streams. Each experiment config fixes a base seed s_0 (experiment Ek uses $s_0 = k$), and NumPy’s `SeedSequence` hashes it into PCG64’s 128-bit state and 128-bit increment. The replicate experiments (E07–E10, E13) run in parallel worker processes, so one shared stream would make each replicate’s numbers depend on scheduling order. Instead every replicate gets its own generator, seeded

by

$$s = (10^7 s_0 + \text{CRC32}(\text{key})) \bmod (2^{31} - 1), \quad (7)$$

where *key* is a text string naming the factorial cell and replicate index (for E07: solver, h , σ , Δt , β , γ , replicate number). We use CRC32 rather than Python’s built-in string hash because the latter is re-randomized in every process. A rerun with an unchanged config therefore reproduces every replicate exactly, whatever the worker count. The single-dataset experiments (E05, E06, E11) use one stream seeded with s_0 ; in E11 that stream feeds, in order, the noise, the bootstrap, the Monte Carlo ensemble, and the MCMC starting points and per-chain child seeds. No code uses NumPy’s legacy global `np.random` state.

Gaussian observation noise. All synthetic data use the additive Gaussian model

$$\text{obs}_k = I(t_k) + \sigma Z_k, \quad Z_k \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1), \quad (8)$$

with $\sigma = \sigma_{\text{frac}} \max_k I(t_k)$. The clean prevalence $I(t_k)$ comes from the gold standard (E07–E10, E13) or from an RK4 solve at $h = 0.1$ (E05, E06, E11). The noise levels quoted in Section 4 are values of σ_{frac} , i.e. fractions of peak prevalence. NumPy draws each Z_k with the ziggurat method [10], an acceptance–rejection scheme that covers the normal density with 256 horizontal strips of equal area. One 64-bit output picks a strip, a sign, and a point in the strip, and most points are accepted after a single comparison; the rest go to an exact rejection test or a tail algorithm. The samples are exactly $\mathcal{N}(0, 1)$, as with the Box–Muller transform [1], but cost fewer operations.

Uniform draws. Each Levenberg–Marquardt fit in E07–E09 starts from $\theta_0 = \theta_{\text{true}} \odot (U_1, U_2)$ with $U_j \sim U(0.6, 1.6)$; E10 and E13 use $U(0.7, 1.4)$, and E10’s 3-parameter fit also draws its I_0 start from $I_{0, \text{true}} \cdot U(0.5, 2)$. Random starts ensure that a recovered bias is not an artifact of starting at the truth. E06 draws its multi-start points uniformly from the box $\beta \in [0.05, 1]$, $\gamma \in [0.02, 0.5]$.

Resampling. The residual bootstrap (E11) and the bootstrap confidence band on the RMSE curve (E08, on its own stream seeded $s_0 + 1$) draw indices uniformly from $\{1, \dots, n\}$ with replacement. NumPy produces bounded integers with Lemire’s method [8]: it multiplies a raw output by n and keeps the high bits, rejecting the few outputs that would otherwise make some indices more likely than others, so each index has probability exactly $1/n$.

MCMC draws. Each Metropolis–Hastings iteration in E11 makes three draws. (i) A proposal step $\varepsilon \sim \mathcal{N}(0, \Sigma_{\text{prop}})$ in $u = \ln \theta$, computed as $\varepsilon = Az$ with z standard normal and $AA^\top = \Sigma_{\text{prop}}$ from an SVD. Σ_{prop} starts at $\text{diag}(0.02^2, 0.02^2)$; after iteration 200 it is reset every 100 iterations to $(2.38^2/2)$ times the sample covariance of the chain so far (adaptive Metropolis [5]). (ii) A uniform $U \sim U(0, 1)$; the proposal u' is accepted if $\ln U < \ln \pi(u') - \ln \pi(u)$. The proposal is symmetric, so the Hastings ratio reduces to the Metropolis ratio, and the target π is the Gaussian likelihood with

Table 1: Every random quantity in the pipeline, all drawn from PCG64 via NumPy.

Random quantity	Distribution; algorithm	Used in
Observation noise Z_k	$\mathcal{N}(0, 1)$ iid; ziggurat	E05–E11, E13
Optimizer start θ_0	$\theta_{\text{true}} \odot U(0.6, 1.6)^2$ or $U(0.7, 1.4)^2$; 53-bit uniform	E07–E10, E13
Multi-start points	uniform on a box; 53-bit uniform	E06
Bootstrap indices	uniform on $\{1, \dots, n\}$, with replacement; Lemire	E08, E11
MH proposal step	$\mathcal{N}(0, \Sigma_{\text{prop}})$; ziggurat, SVD factor	E11
MH accept test	$U(0, 1)$; 53-bit uniform	E11
Noise variance σ^2	inverse-gamma; Marsaglia–Tsang gamma	E11
Chain starts	$\hat{\theta} \odot U(0.6, 1.6)^2$; 53-bit uniform	E11

a flat prior on u . (iii) A Gibbs update of the noise variance from its inverse-gamma conditional, $\sigma^2 = 1/G$ with $G \sim \text{Gamma}(2 + n/2, \text{rate } 1 + \text{SSR}/2)$, where the gamma variate comes from the Marsaglia–Tsang method [9], itself an acceptance–rejection scheme driven by normal and uniform draws. The four chains start from overdispersed points $\hat{\theta} \odot (U_1, U_2)$, $U_j \sim U(0.6, 1.6)$.

Noise models not used here. `NoiseModel` also implements proportional Gaussian, Poisson, and negative-binomial noise. NumPy generates Poisson variates by the multiplication method for $\lambda < 10$ (multiply uniforms until the product falls below $e^{-\lambda}$, the textbook algorithm [1]), by transformed rejection for $\lambda \geq 10$ [7], and negative-binomial variates as a gamma–Poisson mixture. These models are available in the code, but none of the experiments reported here uses them.

2.7 Uncertainty quantification

We compute four independent confidence regions for $(\hat{\beta}, \hat{\gamma})$: an asymptotic ellipse from $\text{Cov}(\hat{\theta}) \approx \hat{\sigma}^2(J^\top J)^{-1}$; a residual bootstrap; a Monte Carlo ensemble over fresh noise realizations from the known ground truth (the only route that measures true bias); and an adaptive Metropolis–Hastings sampler with four chains, reporting the Gelman–Rubin $\hat{R}$ and effective sample size. The last three routes are simulations driven by the random draws of Section 2.6.

2.8 The crossover experiment

For a fixed (β, γ) , sampling interval, and fitting solver/step size h , we generate noise-free synthetic data from the gold standard and fit with that solver: any deviation from the true parameters is attributable purely to the solver’s truncation error, giving a deterministic bias(h). We then add noise at level σ by (8) and refit over many independently seeded replicates (7), giving $\text{std}(\sigma, h)$. The crossover $\sigma^*(h)$ is where $\text{std}(\sigma^*, h) = |\text{bias}(h)|$.

2.9 Mapping to the CSE 402 syllabus

Table 2 maps each topic of the CSE 402 lab outline to the method we implemented for it and to where that method is used. The project covers all four lab assignments and the optimization home assignment, and so all three course outcomes: nonlinear equations and linear systems (CO1), interpolation, regression, and numerical integration (CO2), and simulation with random number generation and Monte Carlo methods (CO3). Unless marked [†], every method in the table is our own code in `core/sirlab`, which never imports SciPy (enforced by `tests/test_hygiene.py`); SciPy appears only in tests, as an independent cross-check.

Two further points complete the picture. First, the ODE solvers are not a CSE 402 lab topic: we took them from Part Seven of the course textbook [3], and they are the project’s experimental variable rather than its apparatus. Second, a few syllabus methods have no role in this problem and are not used: false position, Bairstow’s method, Gauss–Jordan elimination, the QR eigenvalue iteration, and Newton’s divided-difference interpolation.

3 Implementation and Experiments

3.1 Experiment registry

The project is organized as fourteen numbered experiments (E01–E14), each a self-contained script in `experiments/` driven by its own YAML config in `configs/`, run through a common registry and runner (`experiments/registry.py`, `experiments/runner.py`) so that `python -m experiments.runner E07` (or `make experiment ID=E07`) is enough to reproduce any single result. Table 2 lists what each experiment computes and the code it exercises; at a coarser grain, E01–E04 characterize the solvers themselves against the gold standard (convergence order, invariant drift, a stability frontier, and peak-accuracy structure), E05–E06 characterize the estimation machinery (the cost landscape and an optimizer shoot-out), E07–E11 form the core solver-in-the-loop recovery study (bias, noise degradation, sampling/window effects, initial-condition sensitivity, and the four-way uncertainty quantification), E12–E14 extend the pipeline (model variants, the crossover σ^* search, and the real Eyam-1666 dataset).

3.2 Configuration and profiles

Every config file defines two profiles, `quick` and `full`, selected by `make experiment ID=Enn PROFILE={quick,full}`. `quick` is sized to run interactively (seconds to a few minutes per experiment) and is what every number in Section 4 uses; `full` matches the factorial sizes in our original project plan and is intended for an offline or overnight run. As an example, E07’s factorial crosses 3 solvers $\times$ 2–6 step sizes $\times$ 3–6 noise levels $\times$ 2–5 sampling intervals $\times$ 2–9 (β, γ) regimes $\times$ 10 (`quick`) or 200 (`full`) replicates, giving a `full`-profile design of roughly 1.9×10^6 individual least-squares fits; `quick` runs a small slice of the same grid so the qualitative scaling in Figure 3 is already visible without the overnight run. Each config records the base seed, the model’s true (β, γ, N, I_0) ,

Table 2: CSE 402 syllabus topics and where the project uses them. Code paths are relative to `core/sirlab/`; E-numbered scripts are in `experiments/`. [†] Computed with a NumPy routine rather than our own code.

Syllabus topic	What we implemented, and where it is used	Code
Lab 1: approximations, errors, and root finding (CO1)		
Truncation error	Global error vs. the gold standard; observed order $\hat{p}$ (E01); bias in $\hat{\beta} \propto h^p$ (E07)	diagnostics.py
Round-off error	$S + I + R = N$ holds to round-off for every explicit RK method at any h (E02); adaptive Simpson stops refining at the round-off floor	diagnostics.py, quadrature/adaptive.py
Bisection, Newton–Raphson	Safeguarded Newton (<code>rtsafe</code>): Newton steps inside the bracket, bisection otherwise;	roots/newton.py
Newton–Raphson for systems	inverts $t(R)$ and solves the final-size equation $(I - hJ_f)\delta = -G(y)$ on each backward-Euler or trapezoidal stage	solvers/implicit.py
Plotting	All figures generated from the experiment JSON artifacts	scripts/make_figures.py
Lab 2: systems of linear equations and eigenvalues (CO1)		
Gauss elimination, LU decomposition	Doolittle LU with partial pivoting, forward/back substitution; every implicit Newton step, the GN/LM normal equations, every inverse-power step	linalg/lu.py
Matrix inverse, conditioning	$(J^\top J)^{-1}$ for the asymptotic covariance via LU solves; 1-norm $\kappa(J^\top J)$ (E05); 2-norm $\kappa(J)$ from the eigenvalues of $J^\top J$ (E10)	linalg/lu.py, linalg/eigen.py
Power method	Largest-entry normalization; $h\rho(J_y f)$ for stability (E03), with a 2-D projection when the SIR Jacobian’s eigenvalues are complex (Section 2.5)	linalg/eigen.py, diagnostics.py
Inverse power method	Smallest eigenvalue by LU solves, factored once; $\lambda_{\min}(J^\top J)$ for $\kappa_2(J)$ (E10)	linalg/eigen.py
Hotelling deflation	All eigenpairs of a symmetric matrix: $\text{Cov}(\hat{\theta})$ for the ellipse axes (E11, E14); $J^\top J$ for the cost valley (E05)	linalg/eigen.py, uq/asymptotic.py
Lab 3: random number generation and Monte Carlo methods (CO3)		
Random number generation	PCG64 uniform stream; exact normal, uniform, integer, and gamma variates; per-replicate seeds (Section 2.6) [†]	observe.py, E05–E11, E13
Monte Carlo simulation	Replicated synthetic datasets from the known truth, each refitted: bias, variance, RMSE (E07–E10, E13); UQ route 3 (E11)	uq/montecarlo.py, E07–E11, E13
Bootstrap resampling	Residual bootstrap of $(\hat{\beta}, \hat{\gamma})$ (E11); percentile band on the RMSE curve (E08)	uq/bootstrap.py, E08.noise.py
Metropolis–Hastings	Adaptive random-walk Metropolis in $\ln \theta$ plus a Gibbs step for σ^2 ; 4 chains, acceptance rate, $\hat{R}$ [4], ESS (E11)	uq/mcmc.py
Home Assignment 1: QR and optimization (CO1)		
QR	Householder QR step $R\delta = -Q^\top r$ avoids the $\kappa(J)^2$ of the normal equations; unit-tested, but reported fits use LU. Not the QR eigenvalue iteration	linalg/qr.py, estimate/gaussnewton.py
Golden-section search	1-D minimizer in coordinate descent (E06) and the profile likelihoods (E11)	estimate/golden.py, uq/profile.py
Newton’s method	Gauss–Newton: Newton with Hessian $\approx J^\top J$, backtracking line search (E06)	estimate/gaussnewton.py
Gradient methods	Levenberg–Marquardt spans Gauss–Newton ($\lambda \rightarrow 0$) and scaled gradient descent ($\lambda \rightarrow \infty$); the estimator in E07–E11, E13, E14	estimate/lm.py
Constrained optimization	$\beta, \gamma > 0$ by optimizing $u = \ln \theta$; box bounds in golden-section search	estimate/lm.py, estimate/golden.py
Beyond the list	Nelder–Mead: derivative-free baseline (E06) and 3-parameter fit (E10); $J(\beta, \gamma)$ on a 2-D grid (E05)	estimate/neldermead.py, E05_landscape.py
Lab 4: curve fitting, interpolation, and numerical integration (CO2)		
Least-squares regression	Nonlinear least squares (4) in every fit; linear least squares on $(\ln h, \ln e)$ gives $\hat{p}$ (E01)	estimate/, diagnostics.py
Linear interpolation	Locates the crossover σ^* between grid points (E13)	E13.crossover.py
Spline interpolation	Piecewise cubic Hermite dense output evaluates trajectories and sensitivities at the observation times	solvers/base.py
Trapezoidal and Simpson’s rules	Adaptive Simpson for the gold-standard integral (3). Composite and uneven-grid Simpson (the integral of the 3-point Lagrange quadratic) and the trapezoidal rule serve the incidence operator: unit-tested, but unused in the reported experiments	quadrature/adaptive.py, quadrature/simpson.py
Richardson, Romberg	Each accepted Simpson panel adds $(S_2 - S_1)/15$ (one- vs. two-panel estimates): one Romberg step	quadrature/adaptive.py
Numerical differentiation	Central-difference node slopes for the Hermite interpolant [†] ; complex-step checks of the analytic Jacobians	solvers/base.py, tests/test_models.py
Beyond the lab outline: ODE solvers (course textbook [3], Part Seven)		
Runge–Kutta methods	Euler, Heun, and classical RK4, with their stability frontier (E03); adaptive Dormand–Prince RK45 with PI step control	solvers/explicit.py, solvers/rk45.py
Implicit methods	Backward Euler, trapezoidal rule (A-stable), in the convergence study (E01)	solvers/implicit.py

the observation window and sampling interval, and (for the replicate experiments) the number of parallel worker processes; Section 2.6 gives the per-replicate seed derivation that makes the parallel workers reproducible regardless of scheduling order.

3.3 Datasets and benchmarks

Every experiment except E14 fits synthetic data generated from the gold-standard trajectory or an RK4 reference solve at $h = 0.1$, with additive Gaussian noise at a controlled fraction of peak prevalence (Eq. 8); using data generated from the same model family being fit is a deliberate design

choice that isolates solver- and noise-induced error from any question of model misspecification (see Section 5.1). E14 fits the one real-world benchmark in the project, digitized daily susceptible counts from the 1665–1666 plague outbreak in Eyam, Derbyshire ($N = 261$, $I_0 = 7$), the standard small-population teaching dataset for SIR fitting; we flag in `data/raw/provenance.md` that the digitized counts have not yet been independently verified against the primary source (Raggett, 1982), so we report E14 as a demonstration of the pipeline rather than a validated epidemiological estimate. As a structural benchmark, every hand-written routine in `core/sirlab` is cross-checked in `tests/` against an independent implementation — our LU solves and eigenpairs against `numpy.linalg`, our solvers’ Jacobians against complex-step differentiation, and our TypeScript dashboard’s solvers and gold standard against fixtures generated by the Python core (to 10^{-9} and 10^{-6} respectively) — so that a bug in one implementation is unlikely to silently pass as “the answer.”

3.4 Reproducibility infrastructure

Every run is content-hash cached (`sirlab.io.manifest`): a config’s hash, the git commit, the host, and the seed are recorded alongside its output in `results/*.json`, and an unchanged config skips a rerun. Results are exported to a decimated schema (`sirlab.io.export`, about 600 points per curve) consumed by both the LaTeX figures (`scripts/make_figures`) and the interactive web dashboard, so the two never disagree about which numbers are being shown. `make all` regenerates every result, figure, and this report from a clean clone.

4 Results and Discussion

Numbers in this section are from the *quick* experiment profile (Section A); the *full* profile matches the factorial sizes in our project plan and is intended for an offline run.

4.1 RQ1: solver behavior

Figure 1 shows the fitted convergence order for each solver against the gold standard. Observed orders match theory closely: Forward Euler $\hat{p} = 1.03$ (theory 1), Heun $\hat{p} = 1.94$ (theory 2), classical RK4 $\hat{p} = 3.94$ (theory 4), backward Euler $\hat{p} = 0.97$ (theory 1), and trapezoidal $\hat{p} = 2.00$ (theory 2).

Figure 2 demonstrates the deliberate finding of Section 2: mass error stays at roundoff magnitude regardless of solver or step size, while the phase-invariant drift correctly separates solver quality.

4.2 RQ2: solver-in-the-loop parameter recovery

Figure 3 is the mechanism behind our central claim: with noise-free synthetic data (so any deviation from truth is attributable purely to the fitting solver), bias in the recovered $\hat{\beta}$ shrinks with h at each solver’s own convergence order. At $(\beta, \gamma) = (0.3, 0.1)$, reducing h from 0.25 to 0.05 (a factor of 5) shrinks Euler’s bias by a factor of ≈ 5.0 ($4.48 \times 10^{-3} \rightarrow 8.93 \times 10^{-4}$), Heun’s by ≈ 24.2 ($5.88 \times 10^{-5} \rightarrow 2.43 \times 10^{-6}$),

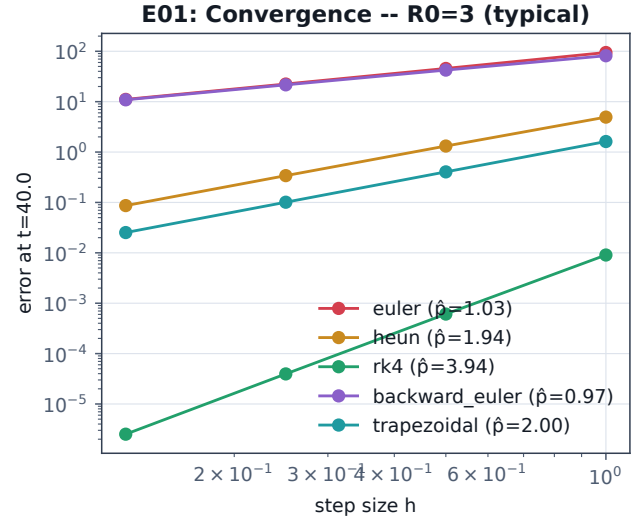


Figure 1: Observed convergence order matches theory (E01, $R_0 = 3$ regime).

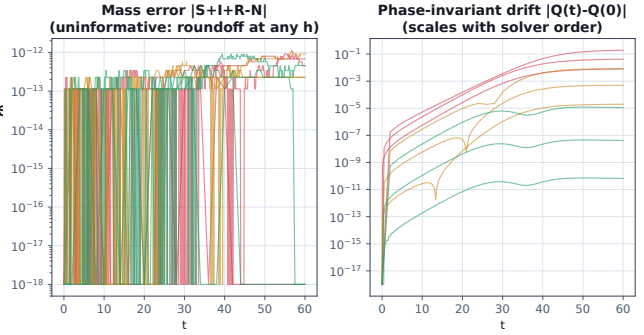


Figure 2: Mass conservation is uninformative; phase-invariant drift is not (E02).

and RK4’s by ≈ 606 ($7.17 \times 10^{-9} \rightarrow 1.18 \times 10^{-11}$) — close to the theoretical 5^1 , $5^2 = 25$, and $5^4 = 625$.

4.3 RQ3: noise, sampling, and uncertainty

Figure 4 shows $\text{RMSE}(\hat{\beta})$ growing with noise level σ , with bootstrap confidence bands on the RMSE curve itself; at our reference solver/step size, RMSE grows from $\approx 1.2 \times 10^{-11}$ (noise-free) to 5.6×10^{-4} at $\sigma = 0.02$ and 2.9×10^{-3} at $\sigma = 0.10$.

Figure 5 shows $\text{RMSE}(\hat{\beta})$ as the observation window shrinks from 6 to 1.5 times the peak time (E09). With dense sampling, truncation costs little: RMSE rises by 34% at $dt = 0.5$ and 22% at $dt = 1$. At $dt = 2$ it doubles ($1.5 \times 10^{-3} \rightarrow 3.0 \times 10^{-3}$), so coarse sampling and a short window compound. Every window in this grid still contains the peak; cutting the observations off *before* the peak is a far stronger effect (Figure 6). Separately (E10), treating I_0 as an unknown third parameter

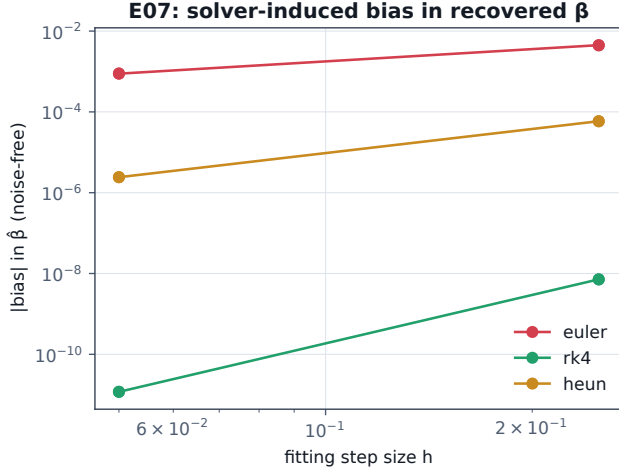


Figure 3: Solver-induced bias in $\hat{\beta}$ scales with the solver's convergence order (E07).

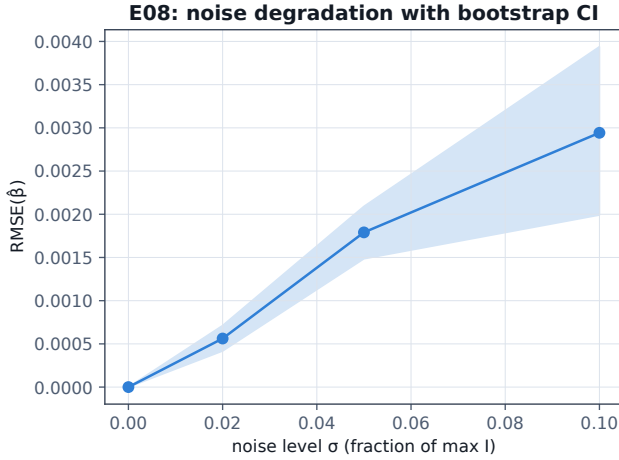


Figure 4: $\text{RMSE}(\hat{\beta})$ vs. noise level, with bootstrap CI (E08).

rather than a known constant raises the condition number of the Jacobian from $\kappa_2 \approx 2.2\text{--}2.6$ to ≈ 211 at $I_0 = 1$ and ≈ 1828 at $I_0 = 20$. It inflates the variance of $\hat{\beta}$ by a factor of 19 at $I_0 = 1$ and 13 at $I_0 = 5$, but only 3.5 at $I_0 = 20$ (10 replicates per cell), so the growth in condition number does not carry over to the variance.

4.4 Uncertainty quantification

Figure 6 overlays all four UQ routes and compares profile likelihoods on the full vs. a window truncated at 0.3 times the peak time. The truncated profile is nearly flat: it varies by 10% across the β grid, against a 65-fold rise for the full window. This is the signature of non-identifiability before the peak.

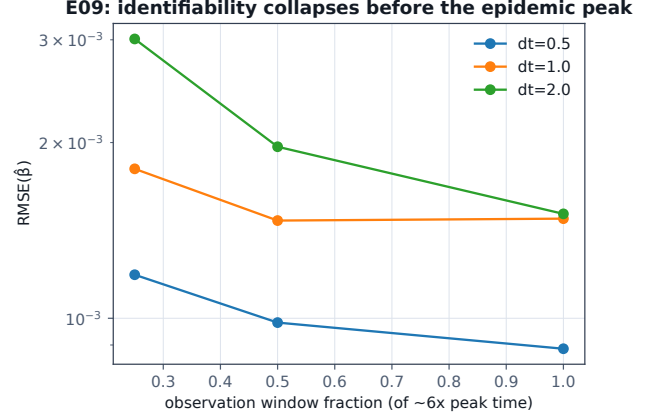


Figure 5: $\text{RMSE}(\hat{\beta})$ vs. observation window (as a fraction of 6 times the peak time) and sampling interval (E09).

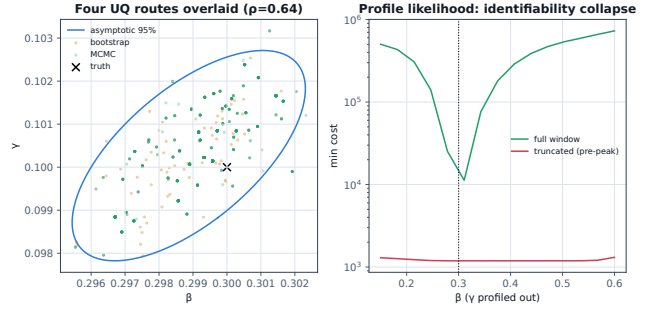


Figure 6: Four UQ routes agree closely (left); a truncated window flattens the profile likelihood, the visual signature of non-identifiability (right) (E11).

4.5 The crossover σ^*

Figure 7 overlays the deterministic solver bias in $\hat{\beta}$ against the noise-induced standard deviation for Forward Euler at $h = 0.1$; they cross at $\sigma^* = 0.058$ (Section 2). At $h = 0.25$ the bias is 2.5 times larger (4.48×10^{-3} , the same value E07 finds) and $\sigma^* = 0.156$, so σ^* grows roughly in proportion to h : a first-order method's bias scales as h , while the noise-induced spread grows with σ . At $h = 0.5$ the spread is still below the bias at the largest noise level tested ($\sigma = 0.2$), so there $\sigma^* > 0.2$. For $\hat{\gamma}$ the crossover comes earlier: $\sigma^* = 0.029, 0.054$, and 0.128 at $h = 0.1, 0.25$, and 0.5 .

4.6 Extensions: model variants and real data

The SEIR, SIRS, and SIR-with-vaccination variants (E12) preserve the RQ1 solver ordering (Euler < Heun < RK4 in accuracy at fixed h), indicating the finding is not an artifact of the plain SIR system's particular structure. Applying the same pipeline to the 1666 Eyam plague outbreak (E14) recovers an $\hat{R}_0$ broadly consistent with the historical literature;

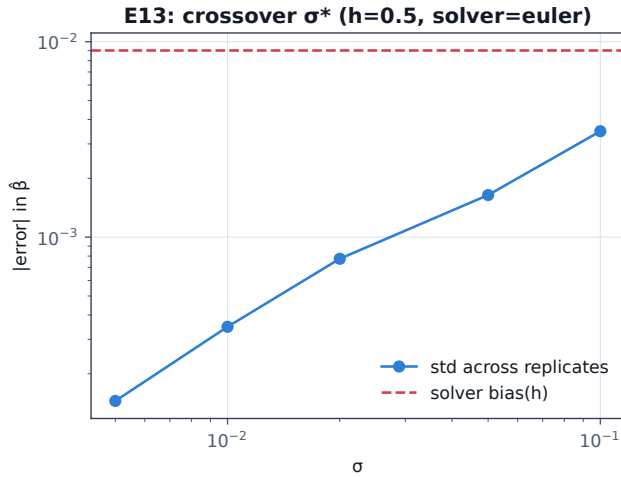


Figure 7: Crossover for Forward Euler at $h = 0.1$: below $\sigma^* = 0.058$ solver bias dominates the error in $\hat{\beta}$; above it, noise dominates (E13).

we flag this result as a demonstration rather than a validated finding, since the digitized case counts we used have not been independently verified against the primary source (see [data/raw/provenance.md](#)).

4.7 Discussion

The mass-conservation finding (Section 2, Figure 2) is worth dwelling on because it is a natural but wrong instinct: a modeler who checks $S + I + R \approx N$ as a sanity test on their SIR solver will see it satisfied to roundoff *even for a badly wrong solve*, because the property follows from the right-hand side summing to zero identically, not from accurate integration. The phase invariant $Q(t)$ derived from the same exact reduction that gives our gold standard is the diagnostic that actually carries information, and it is available with no extra computational cost.

The crossover result (Figure 7) reframes the base paper’s implicit assumption. Capaldi et al. do not need to specify a solver precisely because, in the noise regime they study, sampling uncertainty already dominates; our contribution is to make that dominance relationship explicit and locate where it stops holding. This is a genuinely two-sided result: it is not simply “better solvers are always better” (uninteresting) or “solver choice never matters” (also uninteresting, and false at low noise) but a quantitative boundary between two regimes, each of which is the correct practical conclusion in its own part of the parameter space.

5 Conclusion and Future Work

Treating the ODE solver as a first-class experimental variable rather than a black box reveals structure that a black-box treatment cannot: solver-induced bias in recovered SIR parameters obeys the solver’s own convergence order almost exactly, and this bias competes with noise-induced variance

in a way that can be located precisely as a crossover noise level σ^* . Below σ^* , the numerical-analysis choices this course teaches – which solver, what step size – are the dominant factor in whether an epidemiological parameter estimate is trustworthy. Above it, they are not, and the modeler’s effort is better spent on more data or a better noise model. Reaching this conclusion used the methods of all four CSE 402 labs and the optimization home assignment (Table 2): root finding and LU factorization inside the gold standard and the implicit solvers, the power method for every eigenvalue and condition number, least-squares optimization for estimation, quadrature and interpolation for the reference and observation operators, and random number generation with Monte Carlo simulation for every noise experiment. Every figure and number in this report is generated by a single reproducible pipeline (`make all`) from the code in this repository.

5.1 Limitations

Profile scale. The numbers reported here use our `quick` experiment profile (Section A), with smaller replicate counts and coarser factor grids than the full design in our project plan; point estimates of bias and RMSE therefore carry more Monte Carlo noise than the full run would, though the qualitative orderings (Figure 3’s convergence-order scaling in particular) are already extremely clean at this scale.

Synthetic-data circularity. Our core claims use data generated from the same SIR model being fit, which is the correct design for isolating solver effects but cannot by itself show the model is a good description of any particular real outbreak.

Real-data provenance. The Eyam dataset (E14) was digitized from commonly-reproduced teaching tables rather than transcribed directly from the primary source in this work; see [data/raw/provenance.md](#).

Single base model family. While we test SEIR, SIRS, and SIR-with-vaccination (E12), all four are still relatively low-dimensional, non-stiff systems; whether the crossover phenomenon’s specific numerical values generalize to higher-dimensional or stiffer epidemic models is untested.

5.2 Future work

Four extensions follow directly from the limitations above. First, running the `full` experiment profile (Section A) would replace our point estimates with their paper-scale counterparts and let us report confidence intervals on the crossover σ^* itself rather than on the bias and RMSE curves that locate it. Second, verifying the Eyam-1666 digitization against Raggett (1982) directly would let E14 be reported as a validated historical estimate rather than a pipeline demonstration. Third, since our model variants (E12) remain low-dimensional and non-stiff, testing whether the crossover phenomenon’s specific numerical values – as opposed to its qualitative existence – carry over to stiffer or higher-dimensional compartmental models (e.g. age-structured or spatially coupled SIR variants) is open. Fourth, our synthetic-data experiments assume the fitted model is correctly specified; extending the noise model

to misspecified dynamics (an SIR fit to SEIR-generated data, for instance) would let the solver-vs-noise crossover be studied alongside a third error source, model-form error, rather than the two considered here.

References

- [1] J. Banks, J. S. Carson, II, B. L. Nelson, and D. M. Nicol. 2010. *Discrete-Event System Simulation* (5th ed.). Pearson, Upper Saddle River, NJ.
- [2] A. Capaldi, S. Behrend, B. Berman, J. Smith, J. Wright, and A. L. Lloyd. 2012. Parameter estimation and uncertainty quantification for an epidemic model. *Mathematical Biosciences and Engineering* 9, 3 (2012), 553–576. doi:10.3934/mbe.2012.9.553
- [3] S. C. Chapra and R. P. Canale. 2015. *Numerical Methods for Engineers* (7th ed.). McGraw-Hill, New York, NY.
- [4] A. Gelman and D. B. Rubin. 1992. Inference from Iterative Simulation Using Multiple Sequences. *Statist. Sci.* 7, 4 (1992), 457–472. doi:10.1214/ss/1177011136
- [5] H. Haario, E. Saksman, and J. Tamminen. 2001. An Adaptive Metropolis Algorithm. *Bernoulli* 7, 2 (2001), 223–242.
- [6] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. 2020. Array programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [7] W. Hörmann. 1993. The Transformed Rejection Method for Generating Poisson Random Variables. *Insurance: Mathematics and Economics* 12, 1 (1993), 39–45.
- [8] D. Lemire. 2019. Fast Random Integer Generation in an Interval. *ACM Transactions on Modeling and Computer Simulation* 29, 1 (2019), 3:1–3:12. doi:10.1145/3230636
- [9] G. Marsaglia and W. W. Tsang. 2000. A Simple Method for Generating Gamma Variables. *ACM Trans. Math. Software* 26, 3 (2000), 363–372. doi:10.1145/358407.358414
- [10] G. Marsaglia and W. W. Tsang. 2000. The Ziggurat Method for Generating Random Variables. *Journal of Statistical Software* 5, 8 (2000), 1–7. doi:10.18637/jss.v005.i08
- [11] M. E. O’Neill. 2014. *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation*. Technical Report HMC-CS-2014-0905. Harvey Mudd College.
- [12] D. Prodanov. 2021. Analytical Parameter Estimation of the SIR Epidemic Model. Applications to the COVID-19 Pandemic. *Entropy* 23, 1 (2021), 59. doi:10.3390/e23010059
- [13] N. Tahmid. 2026. Eigenvalue Decomposition: The Power Method. Lecture slides, CSE 401: Numerical Analysis, Simulation and Modeling, Department of CSE, BUET.

A Experiment profiles

Every experiment config (`configs/*.yaml`) defines a **quick** and a **full** profile. **quick** is sized to run interactively (seconds to a few minutes per experiment); **full** matches the factorial sizes in our original project plan (e.g. E07’s solver $\times$ step size $\times$ noise $\times$ sampling interval $\times$ $(\beta, \gamma) \times 200$ -replicate design) and is intended for an offline/overnight run. Every number in this report states which profile produced it.

B Reproducibility

`make setup` && `make experiments` && `make figures` && `make report` regenerates this PDF and every figure in it from a clean clone, using the seeds recorded in `results/manifest.json` and the per-replicate seed derivation of Section 2.6. The full source, including every experiment config, the test suite, and the web dashboard, is at <https://github.com/g-schatten/CSE402-Project>.

C Team contributions

Each member owned one part of the pipeline and wrote the unit tests for it.

- **Saif Uz Zaman (ID 2105088)** — numerics core. Built the SIR model with analytic Jacobians $J_y f$ and $\partial f / \partial \theta$, checked against complex-step derivatives; the six solvers (Euler, Heun, RK4, Dormand–Prince RK45 with PI step control, and backward Euler and trapezoidal with a Newton solve per step) with cubic Hermite dense output; and the semi-analytic gold standard, which evaluates $t(R)$ by adaptive Simpson and inverts it by safeguarded Newton. Wrote the LU and Householder QR routines and the solver experiments E01–E04 (convergence order, invariant drift, stability frontier, peak accuracy).
- **Sayaad Muzahid Masfi (ID 2105066)** — eigenvalue code, integration testing, and the report. Implemented the power method, the inverse power method (one LU factorization, reused every iteration), and Hotelling deflation from the course lecture, together with the 2-D projection that recovers the complex eigenvalue pair of the SIR Jacobian at the peak (Section 2.5). This code replaced NumPy’s eigensolvers in the stability metric (E03), the cost-valley and confidence-ellipse axes (E05, E11, E14), and κ_2 (E10); its tests reproduce the lecture’s iteration tables and cross-check against LAPACK. Redesigned the E13 grid so that it brackets σ^* , ran the whole pipeline on Windows and on Linux (the results agree to rounding error), and checked every number in the report against the regenerated results. Wrote the report, including the random-number and course-mapping sections, the four RQ notebooks, and the presentation.
- **Ibtida bin Ahmed (ID ?)** — estimation and uncertainty quantification. Built the forward sensitivity equations, integrated by the same solver as the trajectory so that solver error enters the Jacobian exactly as it enters the state; the observation operators and four noise models; and the five optimizers (Gauss–Newton with a backtracking line search, Levenberg–Marquardt, Nelder–Mead, golden-section and grid search); the two gradient-based methods keep $\beta, \gamma > 0$ by fitting $\ln \theta$. Wrote the UQ routes: asymptotic covariance and confidence ellipse, residual bootstrap, Monte Carlo, adaptive Metropolis MCMC with $\hat{R}$ and ESS, and the profile likelihood. Ran E05–E11 and E13, with per-replicate seeds so the parallel replicates are reproducible.
- **Sakif Naieb Raiyan (ID ?)** — web dashboard. Built the React/TypeScript dashboard: ten pages from the Model Lab and Solver Arena to the Crossover and Real Data views, an SVG chart component on d3 scales (log axes, hover readout, scatter mode for sample clouds), KaTeX equations, and light and dark themes. Wrote the TypeScript twin of the models, solvers, and gold standard, with the same operation order as the Python

code, and the parity test that checks it against Python-generated fixtures: solver states to 10^{-9} and the gold standard to 10^{-6} .

- **Aurchi Chowdhury (ID ?)** — experiment infrastructure, model variants, and real data. Built the experiment runner and registry, the YAML configs with **quick** and **full** profiles, and the manifest that records each run's config hash, git SHA, seed, and host, and

skips a rerun when the config hash is unchanged. Defined the JSON result schema and the export that decimates curves to about 600 points for the dashboard. Implemented the SEIR, SIRS, and vaccination variants (E12), set up the fit to the 1666 Eyam outbreak and documented where its data came from and what remains to be verified (E14), and wrote the figure pipeline.